

# LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

A.K.A. **AI APPLICATIONS  
IN PRACTICE**

## **2 - Python Data Structures**

Prof. Tiberio Uricchio  
< tiberio.uricchio@unipi.it >

***TUPLES, SETS, DICTIONARIES***

## Recap - Types

Python has an amount of built-in types

|                  |                                                               |
|------------------|---------------------------------------------------------------|
| Numbers          | <code>1234, 3.1415, 3+4j, 0b111, Decimal(), Fraction()</code> |
| Strings          | <code>'spam', "Bob's", b'a\x01c', u'sp\xc4m'</code>           |
| Lists            | <code>[1, [2, 'three'], 4.5], list(range(10))</code>          |
| Dictionaries     | <code>{'food': 'spam', 'taste': 'yum'}, dict(hours=10)</code> |
| Tuples           | <code>(1, 'spam', 4, 'U'), tuple('spam'), namedtuple</code>   |
| Files            | <code>open('eggs.txt'), open(r'C:\ham.bin', 'wb')</code>      |
| Sets             | <code>set('abc'), {'a', 'b', 'c'}</code>                      |
| Other core types | <code>Booleans, types, None</code>                            |

3

## Recap - Types

Python has an amount of built-in types

|                  |                                                               |
|------------------|---------------------------------------------------------------|
| Numbers          | <code>1234, 3.1415, 3+4j, 0b111, Decimal(), Fraction()</code> |
| Strings          | <code>'spam', "Bob's", b'a\x01c', u'sp\xc4m'</code>           |
| Lists            | <code>[1, [2, 'three'], 4.5], list(range(10))</code>          |
| Dictionaries     | <code>{'food': 'spam', 'taste': 'yum'}, dict(hours=10)</code> |
| Tuples           | <code>(1, 'spam', 4, 'U'), tuple('spam'), namedtuple</code>   |
| Files            | <code>open('eggs.txt'), open(r'C:\ham.bin', 'wb')</code>      |
| Sets             | <code>set('abc'), {'a', 'b', 'c'}</code>                      |
| Other core types | <code>Booleans, types, None</code>                            |

4

# Recap – Lists and strings

## Strings

- Sequences of characters
- Immutable

## Lists

- Sequences of objects of any type
- Mutable

5

# Tuples

Another type of immutable objects are **Tuples**.

Tuples are like lists except for the fact that lists are mutable.

Tuples can be defined as comma separated values delimited by parentheses.

```
>>> my_tuple = ('a', 1, 5, 'b')  
  
>>> type(my_tuple)  
  
<class 'tuple'>
```

6

# Tuples

Actually, tuples can also be defined without any delimiter, just as a series of comma separated values.

Usually though, tuples are declared using parenthesis for clarity.

```
>>> my_tuple = 'a', 1, 5, 'b'

>>> type(my_tuple)

<class 'tuple'>
>>> print(my_tuple)
```

7

# Tuples

Operations on tuples are the same that can be done on lists.

```
>>> tuple1 = (1, 4, 'cat', 4.5, 'dog')

>>> tuple2 = (.3, 9, None, 'bird')

>>> tuple3 = tuple1 + tuple2

>>> type(tuple3)

<class 'tuple'>
>>> print(tuple3)

(1, 4, 'cat', 4.5, 'dog', 0.3, 9, None, 'bird')
```

8

# Tuples

Operations on tuples are the same that can be done on lists.

```
>>> print(tuple3)
(1, 4, 'cat', 4.5, 'dog', 0.3, 9, None, 'bird')
>>> tuple3*2
(1, 4, 'cat', 4.5, 'dog', 0.3, 9, None, 'bird', 1, 4, 'cat', 4.5, 'dog', 0.3, 9, None, 'bird')
```

9

# Tuples

Operations on tuples are the same that can be done on lists...

... Except assignment!

```
>>> my_list = ['1', 'abc', 5, 2.3]
>>> print(my_list)
['1', 'abc', 5, 2.3]
>>> print(my_list[2])
5
>>> my_list[2] = 9999
>>> print(my_list)
['1', 'abc', 9999, 2.3]

>>> my_tuple = ('1', 'abc', 5, 2.3)
>>> print(my_tuple)
('1', 'abc', 5, 2.3)
>>> print(my_tuple[2])
5
>>> my_tuple[2] = 9999
Traceback (most recent call last):
  File "<pyshell#374>", line 1, in <module>
    my_tuple[2] = 9999
TypeError: 'tuple' object does not support item assignment
```

10

# Tuples - Packing

When declaring a tuple we are basically assigning multiple objects to the same variable. This operation is called *packing*.

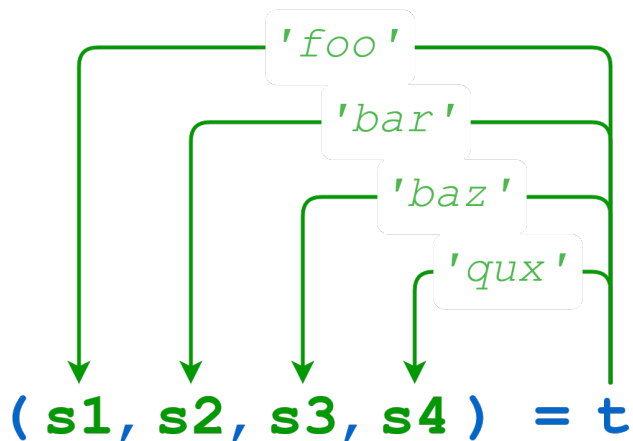
```
>>> t = ('foo', 'bar', 'baz', 'qux')
```

11

# Tuples - Unpacking

The opposite operation is also possible. With *unpacking* a set of objects can be assigned to multiple variables.

```
>>> (s1, s2, s3, s4) = t
>>> print(s1)
foo
>>> print(s2)
bar
>>> print(s3)
baz
>>> print(s4)
qux
```



12

# Tuples - Unpacking

When unpacking, the number of variables on the left hand side of the assignment must match the number of objects in the tuple.

```
>>> print(t)

('foo', 'bar', 'baz', 'qux')
>>> s1, s2, s3 = t

Traceback (most recent call last):
  File "<pyshell#407>", line 1, in <module>
    s1, s2, s3 = t
ValueError: too many values to unpack (expected 3)
```

13

# Tuples - Unpacking

When unpacking, the number of variables on the left hand side of the assignment must match the number of objects in the tuple.

```
>>> print(t)

('foo', 'bar', 'baz', 'qux')
>>> s1, s2, s3, s4, s5 = t

Traceback (most recent call last):
  File "<pyshell#411>", line 1, in <module>
    s1, s2, s3, s4, s5 = t
ValueError: not enough values to unpack (expected 5, got 4)
```

14

## Swapping variables

```
>>> a = 1
>>> b = 2
>>> tmp = a
>>> a = b
>>> b = tmp
>>> print(a)
2
>>> print(b)
1
```

To swap variables we usually need a temporary variable to store one of the values.

15

## Swapping variables

```
>>> a = 1
>>> b = 2
>>> b, a = a, b
>>> print(a)
2
>>> print(b)
1
```

Thanks to packing/unpacking, variables can simply be swapped without any intermediate step.

16

## Swapping variables

```
>>> a = 1
>>> b = 2
>>> b, a = a, b
>>> print(a)
2
>>> print(b)
1
```

Thanks to packing/unpacking, variables can simply be swapped without any intermediate step.

Variables «a» and «b» are packed inside a tuple

17

## Swapping variables

```
>>> a = 1
>>> b = 2
>>> b, a = a, b
>>> print(a)
2
>>> print(b)
1
```

Thanks to packing/unpacking, variables can simply be swapped without any intermediate step.

The objects are then unpacked inside «b» and «a»

18

# Lists and tuples

What are the types of the objects contained in these variables?

```
>>> a = [1, 2, 3, 4]
>>> b = (1, 2, 3, 4)
>>> c = [3, (1, 2, 3), 'abc']
>>> d = (3, [1, 2, 3], 'abc')
>>> e = [(1, 2, 3, 4)]
>>> f = ([1, 2, 3, 4])
```

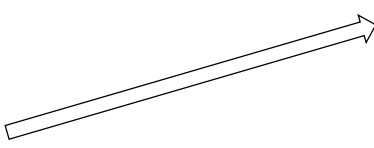
19

# Lists and tuples

What are the types of the objects contained in these variables?

```
>>> a = [1, 2, 3, 4]
>>> b = (1, 2, 3, 4)
>>> c = [3, (1, 2, 3), 'abc']
>>> d = (3, [1, 2, 3], 'abc')
>>> e = [(1, 2, 3, 4)]
>>> f = ([1, 2, 3, 4])
```

```
>>> type(a)
<class 'list'>
```



20

# Lists and tuples

What are the types of the objects contained in these variables?

```
>>> a = [1, 2, 3, 4]
```

```
>>> b = (1, 2, 3, 4)
```

```
>>> c = [3, (1, 2, 3), 'abc']
```

```
>>> d = (3, [1, 2, 3], 'abc')
```

```
>>> e = [(1, 2, 3, 4)]
```

```
>>> f = ([1, 2, 3, 4])
```

```
>>> type(a)
```

```
<class 'list'>
```

```
>>> type(b)
```

```
<class 'tuple'>
```

21

# Lists and tuples

What are the types of the objects contained in these variables?

```
>>> a = [1, 2, 3, 4]
```

```
>>> b = (1, 2, 3, 4)
```

```
>>> c = [3, (1, 2, 3), 'abc']
```

```
>>> d = (3, [1, 2, 3], 'abc')
```

```
>>> e = [(1, 2, 3, 4)]
```

```
>>> f = ([1, 2, 3, 4])
```

```
>>> type(a)
```

```
<class 'list'>
```

```
>>> type(b)
```

```
<class 'tuple'>
```

```
>>> type(c)
```

```
<class 'list'>
```

22

# Lists and tuples

What are the types of the objects contained in these variables?

```
>>> a = [1, 2, 3, 4]
>>> b = (1, 2, 3, 4)
>>> c = [3, (1, 2, 3), 'abc']
>>> d = (3, [1, 2, 3], 'abc')
>>> e = [(1, 2, 3, 4)]
>>> f = ([1, 2, 3, 4])
```

```
>>> type(a)
<class 'list'>
>>> type(b)
<class 'tuple'>
>>> type(c)
<class 'list'>
>>> type(d)
<class 'tuple'>
```

23

# Lists and tuples

What are the types of the objects contained in these variables?

```
>>> a = [1, 2, 3, 4]
>>> b = (1, 2, 3, 4)
>>> c = [3, (1, 2, 3), 'abc']
>>> d = (3, [1, 2, 3], 'abc')
>>> e = [(1, 2, 3, 4)]
>>> f = ([1, 2, 3, 4])
```

```
>>> type(a)
<class 'list'>
>>> type(b)
<class 'tuple'>
>>> type(c)
<class 'list'>
>>> type(d)
<class 'tuple'>
>>> type(e)
<class 'tuple'>
>>> type(f)
<class 'list'>
```

24

# Lists and tuples

What are the types of the objects contained in these variables?

```
>>> a = [1, 2, 3, 4]
```

```
>>> b = (1, 2, 3, 4)
```

```
>>> c = [3, (1, 2, 3), 'abc']
```

```
>>> d = (3, [1, 2, 3], 'abc')
```

```
>>> e = [(1, 2, 3, 4)]
```

```
>>> f = ([1, 2, 3, 4])
```

```
>>> type(a)
```

```
<class 'list'>
```

```
>>> type(b)
```

```
<class 'tuple'>
```

```
>>> type(c)
```

```
<class 'list'>
```

```
>>> type(d)
```

```
<class 'tuple'>
```

```
>>> type(e)
```

```
<class 'list'>
```

```
>>> type(f)
```

```
<class 'list'>
```

5

## One element tuples

We said that tuples can be defined with or without parentheses.

What actually defines the tuple is the commas that separate the values.

To make a 1-element tuple, we just need to add a comma after the element.

```
>>> a = (1)
```

```
>>> type(a)
```

```
<class 'int'>
```

```
>>> a = (1,)
```

```
>>> type(a)
```

```
<class 'tuple'>
```

```
>>> a = 1,
```

```
>>> type(a)
```

```
<class 'tuple'>
```

# Empty tuples

At the same time, an empty tuple can be defined only by using parentheses, without commas...

```
>>> a = (,)

SyntaxError: invalid syntax
>>> a = ,

SyntaxError: invalid syntax
>>> a = ()

>>> type(a)

<class 'tuple'>
```

27

# ZIP

The **zip()** function takes iterables and returns an iterator of tuples, packing the i-th element from all the iterables.

```
1 numbers = [1, 2, 3, 4, 5, 6, 7]
2 letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
3
4 z = zip(letters, numbers)
5
6 for i in z:
7     print('i: {}, number: {}, letter: {}'.format(i, i[0], i[1]))
```

Shell ×

```
>>> %Run zip_example.py
```

```
i: ('a', 1), number: a, letter: 1
i: ('b', 2), number: b, letter: 2
i: ('c', 3), number: c, letter: 3
i: ('d', 4), number: d, letter: 4
i: ('e', 5), number: e, letter: 5
i: ('f', 6), number: f, letter: 6
i: ('g', 7), number: g, letter: 7
```

28

# ZIP

When the iterables passed to `zip()` have different lengths, the iterator will contain a number of tuples equal to the shortest iterable, excluding all elements of the iterables that exceed that length.

```
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
2 letters = ['a', 'b', 'c', 'd']
3
4 z = zip(letters, numbers)
5
6 for i in z:
7     print('i: {}, number: {}, letter: {}'.format(i, i[0], i[1]))
```

Shell ×

```
>>> %Run zip_example.py
i: ('a', 1), number: a, letter: 1
i: ('b', 2), number: b, letter: 2
i: ('c', 3), number: c, letter: 3
i: ('d', 4), number: d, letter: 4
>>>
```

29

# ZIP

Tuples generated by the `zip` function can be directly unpacked in for loops.

```
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
2 letters = ['a', 'b', 'c', 'd']
3
4 z = zip(letters, numbers)
5
6 for i, j in z:
7     print('number: {}, letter: {}'.format(i, j))
8
```

Shell ×

```
>>> %Run zip_example.py
number: a, letter: 1
number: b, letter: 2
number: c, letter: 3
number: d, letter: 4
```

30

# Tuples - Exercises

- Given a tuple of ints, remove the char and substitute with the missing int.  
tup = (1,2,3,'s',5,6,7)
- Repeat the previous example but this time you don't know where the char is. tup = (1,2,3,'s',5,'b',7,'k',9,10,'d',12)
- Create a new tuple with each i-th element containing the sum of the i-th elements of the original tuples:
  - Tup1 = (3,6,9)
  - Tup2 = (1,1,1)
  - Tup3 = (6,3,0)

31

# Sets

A **set** is a data structure equivalent to sets in mathematics.

A set may contain different types of *immutable* objects without repetitions and without an order → elements in a set are **unordered** and **unique**.

Sets are **mutable**.

Sets are **iterable**.

A set is defined by enclosing its elements in curly brackets { }.

```
>>> a = {1, 2, 'b', 4.3}
```

```
>>> type(a)
```

```
<class 'set'>
```

```
>>> print(a)
```

```
{'b', 1, 2, 4.3}
```

32

# Sets

Do sets support indexing or slicing?

33

# Sets

Do sets support indexing or slicing? **NO**

Sets do not support indexing or slicing since they are unordered.

There is no point in looking for the i-th element in an unordered collection.

```
>>> a = {1, 4, 2, 9}
>>> print(a)
{1, 2, 4, 9}
>>> a[0]
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    a[0]
TypeError: 'set' object does not support indexing
```

34

# Sets

Even if you cannot index a set, sets are mutable since you can modify its elements.

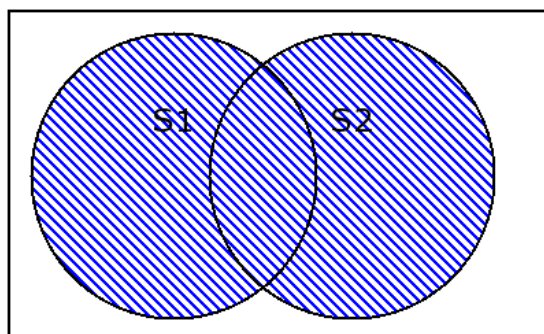
```
>>> a = {1, 3, 9}
>>> a.add(15)
>>> a.add('abc')
>>> print(a)
{1, 3, 9, 15, 'abc'}

>>> print(a)
{1, 3, 9, 15, 'abc'}
>>> a.remove(15)
>>> print(a)
{1, 3, 9, 'abc'}
```

35

## Operations between sets

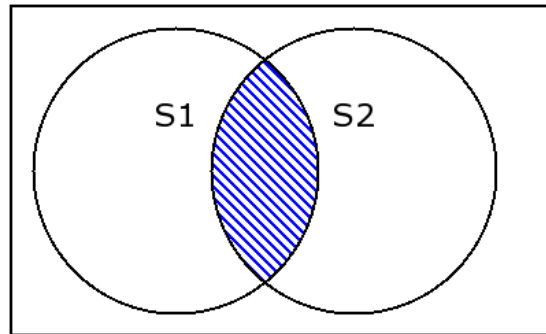
s1.union(s2)



36

## Operations between sets

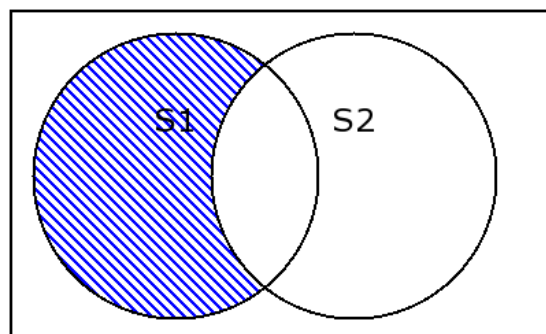
`s1.intersection(s2)`



37

## Operations between sets

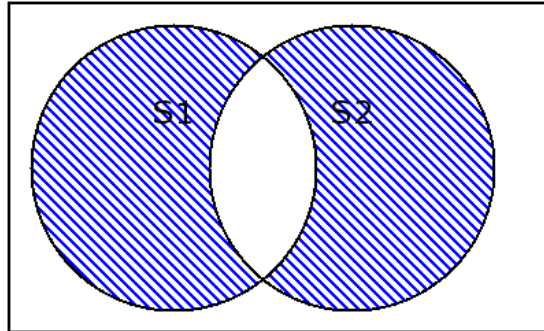
`s1.difference(s2)`



38

# Operations between sets

`s1.symmetric_difference(s2)`



39

## Sets - operations

|                                                          |                                                                                                                                                               |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A   B</b><br><b>A.union(B)</b>                        | Returns a set which is the union of sets <b>A</b> and <b>B</b> .                                                                                              |
| <b>A  = B</b><br><b>A.update(B)</b>                      | Adds all elements of array <b>B</b> to the set <b>A</b> .                                                                                                     |
| <b>A &amp; B</b><br><b>A.intersection(B)</b>             | Returns a set which is the intersection of sets <b>A</b> and <b>B</b> .                                                                                       |
| <b>A &amp;= B</b><br><b>A.intersection_update(B)</b>     | Leaves in the set <b>A</b> only items that belong to the set <b>B</b> .                                                                                       |
| <b>A - B</b><br><b>A.difference(B)</b>                   | Returns the set difference of <b>A</b> and <b>B</b> (the elements included in <b>A</b> , but not included in <b>B</b> ).                                      |
| <b>A -= B</b><br><b>A.difference_update(B)</b>           | Removes all elements of <b>B</b> from the set <b>A</b> .                                                                                                      |
| <b>A ^ B</b><br><b>A.symmetric_difference(B)</b>         | Returns the symmetric difference of sets <b>A</b> and <b>B</b> (the elements belonging to either <b>A</b> or <b>B</b> , but not to both sets simultaneously). |
| <b>A ^= B</b><br><b>A.symmetric_difference_update(B)</b> | Writes in <b>A</b> the symmetric difference of sets <b>A</b> and <b>B</b> .                                                                                   |
| <b>A &lt;= B</b><br><b>A.issubset(B)</b>                 | Returns <b>true</b> if <b>A</b> is a subset of <b>B</b> .                                                                                                     |
| <b>A &gt;= B</b><br><b>A.issuperset(B)</b>               | Returns <b>true</b> if <b>B</b> is a subset of <b>A</b> .                                                                                                     |
| <b>A &lt; B</b>                                          | Equivalent to <b>A &lt;= B</b> and <b>A != B</b>                                                                                                              |
| <b>A &gt; B</b>                                          | Equivalent to <b>A &gt;= B</b> and <b>A != B</b>                                                                                                              |

40

# Sets comprehensions

There is also an equivalent of list comprehensions for sets

```
>>> {s for s in [1, 2, 1, 0]}
set([0, 1, 2])

>>> {s**2 for s in [1, 2, 1, 0]}
set([0, 1, 4])

>>> {s**2 for s in range(10)}
set([0, 1, 4, 9, 16, 25, 36, 49, 64, 81])

>>> {(m, n) for n in range(2) for m in range(3, 5)}
set([(3, 0), (3, 1), (4, 0), (4, 1)])
```

41

# Sets cast

A list can be casted to a set and viceversa. The elements of the list should be immutable and, in case of duplicate elements, they are kept only one.

```
# List with duplicate elements
my_list = [1, 2, 3, 2, 1, 4]

# Converting list to set (removes duplicates)
my_set = set(my_list)
print(my_set) # Output: {1, 2, 3, 4}

# Converting back to list
new_list = list(my_set)
print(new_list) # Output: [1, 2, 3, 4]

# Attempting to use mutable elements will cause an error
# This will raise TypeError: unhashable type: 'list'
# bad_set = set([[1, 2], [3, 4]])
```

42

## Sets - Exercises

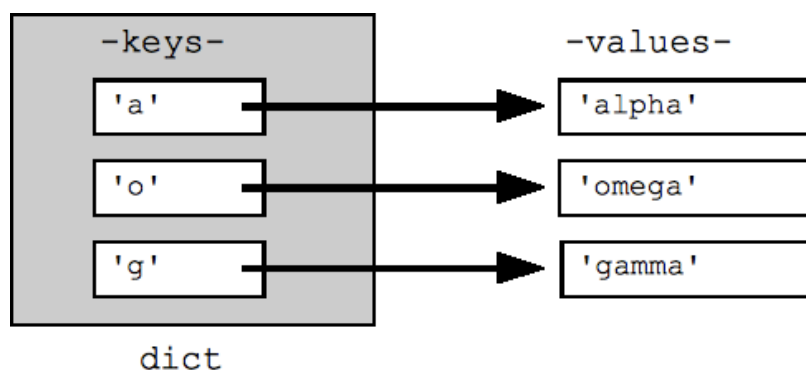
- Taking the string “Hello World” print the count of unique characters, ignoring case and spaces. It should return 7 unique characters. Use set operations to solve this efficiently.
- Given a list of words, use set comprehension to create a set containing only words that have no repeated letters. For example, from ["hello", "python", "set", "unique"], only "python" and "set" would be included.

43

## Dictionaries

Similarly to sets, dictionaries are a collection of unordered elements.

What distinguishes dictionaries is that its elements (the **values**) are indexable with other elements called **keys**.



44

# Dictionaries

Dictionaries can be defined using curly brackets and specifying the elements like:

**'key': value**

Dictionaries cannot be indexed as lists, but only with the keys

```
>>> d = {'one': 1, 'two': 2, 'three': 3}    >>> d = {'one': 1, 'two': 2, 'three': 3}
>>> type(d)                                >>> print(d['two'])
<class 'dict'>                             2
>>> print(d)                                >>> print(d['three'])
{'one': 1, 'two': 2, 'three': 3}            3
>>> d[1]
Traceback (most recent call last):
  File "<pyshell#50>", line 1, in <module>
    d[1]
KeyError: 1
```

45

# Dictionaries

Dictionaries can be defined using curly brackets and specifying the elements like:

**'key': value**

Dictionaries cannot be indexed as lists, but only with the keys

```
>>> d = {'one': 1, 'two': 2, 'three': 3}    >>> d = {'one': 1, 'two': 2, 'three': 3}
>>> type(d)                                >>> print(d['two'])
<class 'dict'>                             2
>>> print(d)                                >>> print(d['three'])
{'one': 1, 'two': 2, 'three': 3}            3
>>> d[1]
Traceback (most recent call last):
  File "<pyshell#50>", line 1, in <module>
    d[1]
KeyError: 1
```

**Dictionaries are associative arrays  
since they map keys to values**

46

# Dictionaries

Keys are unique in a dictionary, but values may not be.

The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

47

# Dictionaries

Elements in a dictionary can be accessed and updated

```
>>> student = {'name': 'Marco', 'age': 25}
>>> print(student)
{'name': 'Marco', 'age': 25}
>>> student['age'] = 27
>>> print(student)
{'name': 'Marco', 'age': 27}
```

48

# Dictionaries

You can also access elements using the **get()** function

```
>>> d = {'a': 1, 'b': 2, 'c': 3}
>>> print(d)
{'a': 1, 'b': 2, 'c': 3}
>>> d.get('b')
2
```

49

# Dictionaries

The **get()** function returns None if the element is not present in the keys. The second argument can be specified to override the returned value in case the key is not present.

```
# Creating a dictionary
student = {'name': 'Alice', 'age': 22, 'major': 'Computer
Science'}

# Using get() with existing key
print(student.get('name')) # Output: Alice

# Using get() with non-existing key (returns None by default)
print(student.get('address')) # Output: None

# Using get() with a default value for non-existing key
print(student.get('address', 'Not Available')) # Output: Not
Available

# Compare with direct dictionary access which raises KeyError
# print(student['address']) # Would raise KeyError: 'address'
```

50

# Dictionaries

All keys in a dictionary can be retrieved using the function **keys()**

```
>>> d = {'a': 1, 'b': 2, 'c': 'three'}
>>> d.keys()
dict_keys(['a', 'b', 'c'])
```

51

# Dictionaries

All keys in a dictionary can be retrieved using the function **keys()**

We said before that dictionaries are a collection of unordered items, like sets.

This is not completely true.

```
>>> d = {'a': 1, 'b': 2, 'c': 'three'}
>>> d.keys()
dict_keys(['a', 'b', 'c'])
```

Since Python 3.7 the keys of a dictionary are stored in creation order.

Therefore the **keys()** function will always return the keys in the same order.

52

# Dictionaries

All elements in a dictionary can be retrieved using the function **values()**

```
>>> d = {'a': 1, 'b': 2, 'c': 'three'}
>>> d.values()
dict_values([1, 2, 'three'])
```

53

# Dictionaries

Keys and values can be obtained together, zipped in tuples, using the function **items()**

```
>>> d = {'a': 1, 'b': 2, 'c': 'three'}
>>> d.items()
dict_items([('a', 1), ('b', 2), ('c', 'three')])
```

54

# Dictionaries

The function **pop()** removes a key (and its value) from the dictionary. The function also returns the object that has been removed.

```
>>> d = {'a': 1, 'b': 2, 'c': 3}
>>> print(d)
{'a': 1, 'b': 2, 'c': 3}
>>> d.pop('b')
2
>>> print(d)
{'a': 1, 'c': 3}
```

55

# Dictionaries comprehensions

There is also an equivalent of list comprehensions for dictionaries

```
>>> {k: v for k, v in [(1, 2), (3, 4)]}
{1: 2, 3: 4}
>>> {n: n for n in range(2)}
{0: 0, 1: 1}
>>> {chr(n): n for n in (65, 66, 66)}
{'A': 65, 'B': 66}
>>> {k: v for k, v in (('I', 1), ('II', 2))}
{'I': 1, 'II': 2}
```

56

# Exercises

- Given a key, check whether a dictionary already contains the key.
- Write a Python script to print a dictionary where the keys are numbers between 1 and 15 (both included) and the values are square of keys.
- Given a set of mixed types, create a list that contains all the types without repetition.